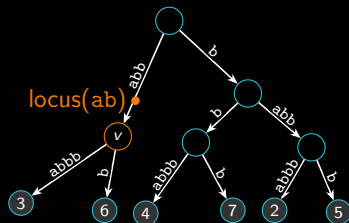


Smallest suffixient set maintenance in near-real-time

Dominik Köppl¹ Gregory Kucherov²

¹: University of Yamanashi, Computer Science and Engineering

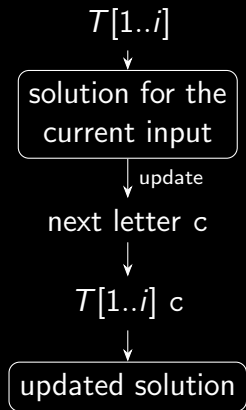
²: Université Gustave Eiffel



background

- many algorithms are **offline**:
compute the solution for the entire input
- while reading the input, we may want to know the solution for the **part read so far**
- i.e., at each point in time, for the text $T[1..i]$, we want to inspect a **snapshot** of the solution currently being maintained
- in an **online algorithm**, whenever a new letter arrives, the previous solution is updated efficiently

Here, an *intermediate solution* is not an approximate solution, but the *complete* solution for the prefix received so far.



online algorithms

Definition (online algorithm)

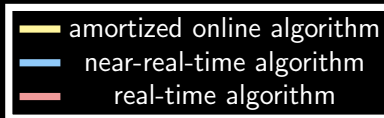
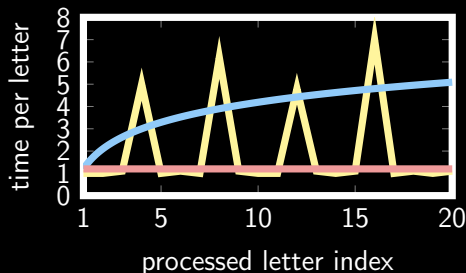
update the current solution incrementally as new letters arrive

however:

- most online algorithms are **amortized**: worst-case time per letter may be high!
- real-time** algorithm: worst-case time per letter is a *constant*

here:

- near-real-time** algorithm: worst-case time per letter is *polyloglogarithmic* in the input size



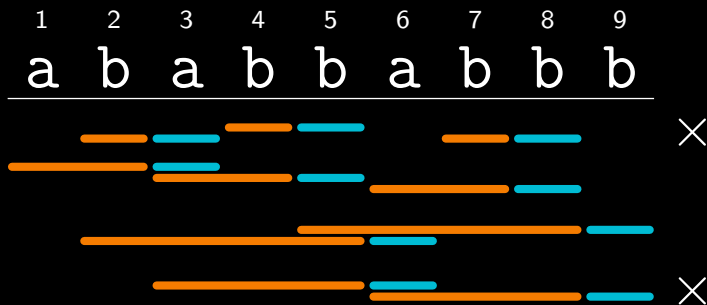
right extension

- ▮ $T[1..n]$: string over an alphabet Σ with size σ
- ▮ given a substring S of T , $x, y \in \Sigma$, $x \neq y$
- ▮ if $S \cdot x$ and $S \cdot y$ are substrings of T ,
then $S \cdot x$ and $S \cdot y$ are **right extensions** (RE)



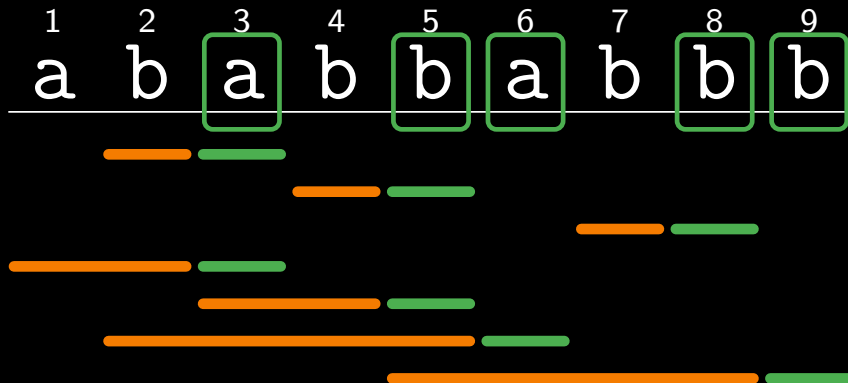
supermaximal right extension

- given an RE $S \cdot x$
 - $S \cdot x$ is a **supermaximal right extension (SRE)**
if $S \cdot x$ is not a suffix of any other RE
- $\Rightarrow$ every RE is a suffix of some SRE



suffixient set

- ▮ P : set of positions in $[1..n]$ (recall: $T[1..n]$ has length n)
- ▮ P is a **suffixient set** if it contains the end position of at least one occurrence of each SRE



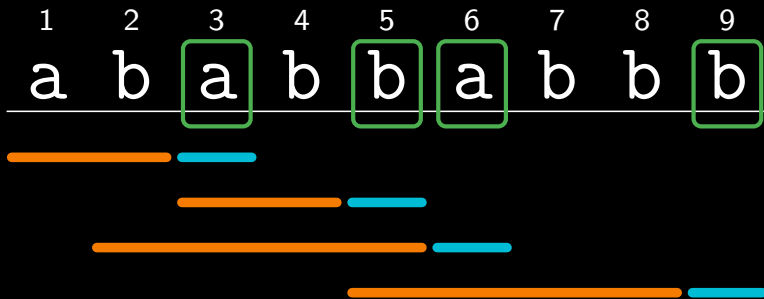
why suffixient sets matter

- suffixient sets support compressed indexing [Depuydt+'23]
- Shibata–Bannai'26 showed how to losslessly represent a text in $O(|SRE|)$ words
- (side note: suffixient sets are string attractors)

indexing + compression

smallest suffixient set (SSS)

- ▮ $\chi(T) :=$ size of a smallest suffixient set (SSS)
- ▮ $\chi(T) = |\text{SRE}(T)|$ (number of SREs in T)
- ▮ construction: for each SRE, choose one end position of one of its occurrences



problem studied here

input

- ▮ string $T[1..n]$ of alphabet Σ read online
- ▮ letters arrive one by one from right to left: $T \leftarrow xT$

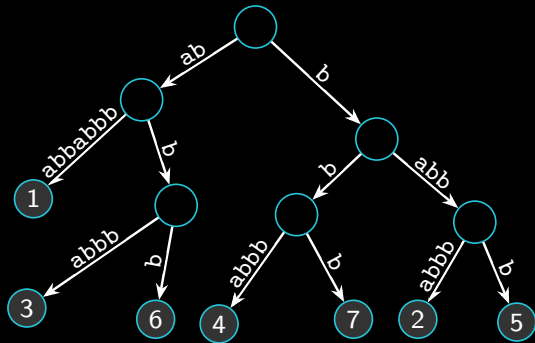
maintain

- ▮ the suffix tree $ST(T)$
- ▮ the SRE set $SRE(T)$
- ▮ an SSS consisting of rightmost end positions

goal: maintain SRE/SSS within the time of one suffix-tree update

suffix tree

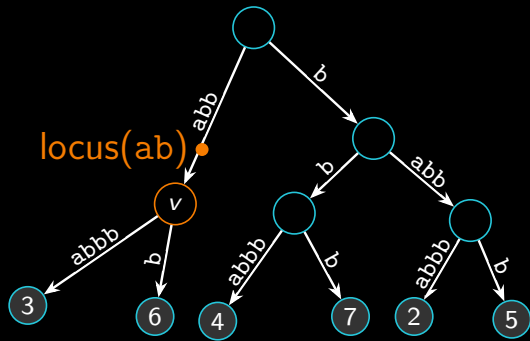
- **suffix tree**: compact trie of all suffixes of $T = \text{ababbabb}$
- (unique terminal symbol \$ omitted from the pictures)



$T =$ 1 2 3 4 5 6 7 8 9
a b a b b a b b b

right-to-left input and Weiner's suffix tree

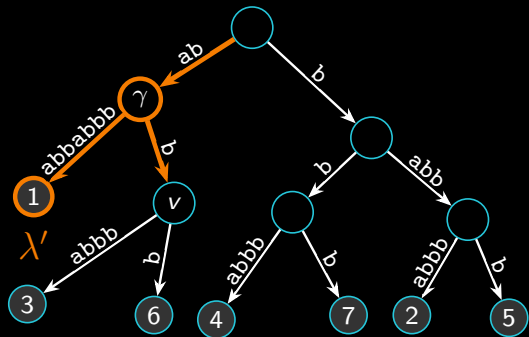
- suppose that we already maintain $ST(T[2..])$
- prepend the new letter $T[1]$ and update to $ST(T[1..])$
- split the edge containing $\text{locus}(ab)$, whose closest descendant node is v



$T =$ 1 2 3 4 5 6 7 8 9
 a b a b b a b b b

right-to-left input and Weiner's suffix tree

- suppose that we already maintain $ST(T[2..])$
- prepend the new letter $T[1]$ and update to $ST(T[1..])$
- split the edge containing $\text{locus}(ab)$, whose closest descendant node is v
- create a new internal node $\gamma = \text{locus}(ab)$, attach new leaf λ' and v as children

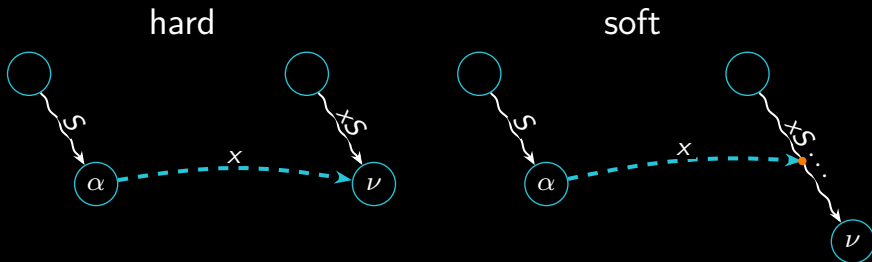


$T =$

1	2	3	4	5	6	7	8	9
a	b	a	b	b	a	b	b	b

definition: Weiner links

- Given node α with $\text{locus}(S) = \alpha$
- If $\text{locus}(x \cdot S)$ exists, then, for this talk, $W_x(\alpha)$ points to $\text{locus}(x \cdot S)$
- $W_x(\alpha)$ is called the x -link of α
- $W_x(\alpha)$ is **hard** if $\text{locus}(x \cdot S)$ is a node
- $W_x(\alpha)$ is **soft** if $\text{locus}(x \cdot S)$ lies inside an edge



suffix tree update time complexity t_{ST}

naive approach:

- ▀ traverse $ST(T[j+1..])$ from root to match $T[j..]$
- ▀ $O(n)$ time

Can we do better?

t_{ST} (unqualified bounds are worst-case)	reference
$O(\lg \sigma)$ amortized	Weiner'73
$O(\lg n)$	Amir+'05
$O(\sigma \lg \lg n)$	Breslauer, Italiano'13
$O(\lg \lg n + \lg \lg \sigma)$ expected	Kopelowitz'12
$O(\lg \lg n + \frac{\lg^2 \lg \sigma}{\lg \lg \lg \sigma})$	Fischer, Gawrychowski'15
$O(\lg \lg n)$ for $\sigma = O(\lg^{1/4} n)$	Kuchеров, Nekrich'17

- ▀ sophisticated data structures make a Weiner update near-real-time
- ⇒ our goal is to preserve this bound when maintaining SREs

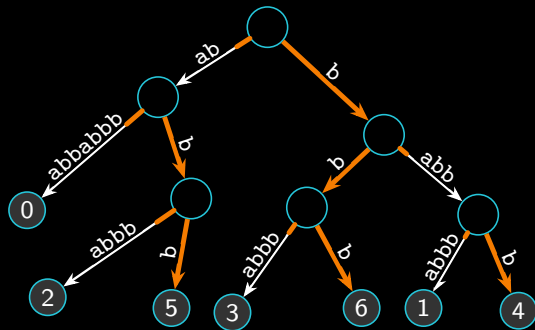
REs and suffix trees

- REs are exactly the set of path labels from the root to the **first letter** of each suffix-tree edge
- We call an edge starting with letter x an x -edge

Lemma

Let $S \cdot y$ be a substring of T .
Then $S \cdot y$ is an RE iff:

- $\text{locus}(S)$ is a node α , and
- α has a y -edge



characterizing SREs in the suffix tree

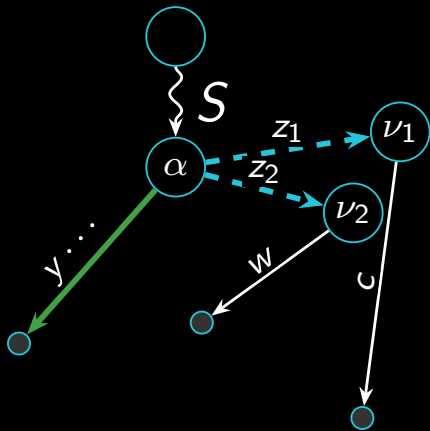
Theorem

Let $S \cdot y$ be a substring of T .

Then $S \cdot y$ is an SRE iff:

- ▮ $\text{locus}(S)$ is a node α ,
- ▮ α has a y -edge, and
- ▮ for every hard link $W_z(\alpha) = \nu$, the node ν has no y -edge.

shortcut: write $S \cdot y$ as (α, y)

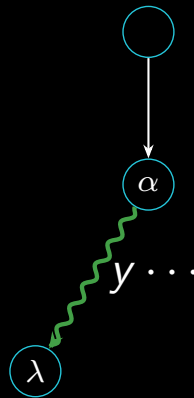


nodes obtained in one Weiner round

setting: update T to $x \cdot T$

▀ λ : newest leaf of old T

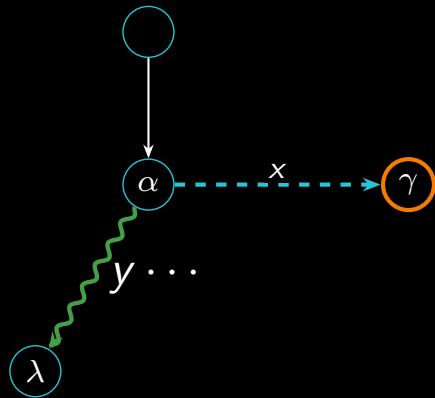
▀ α : lowest ancestor of λ with an x -link



nodes obtained in one Weiner round

setting: update T to $x \cdot T$

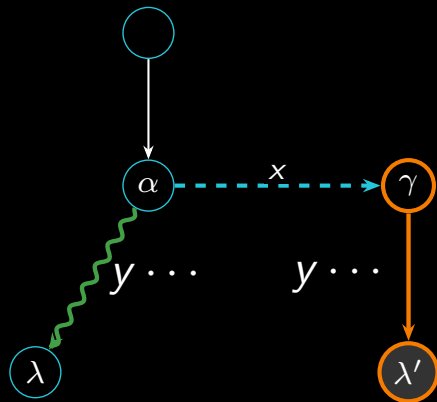
- ▀ λ : newest leaf of old T
- ▀ α : lowest ancestor of λ with an x -link
- ▀ Weiner's algorithm finds α and $\gamma = W_x(\alpha)$



nodes obtained in one Weiner round

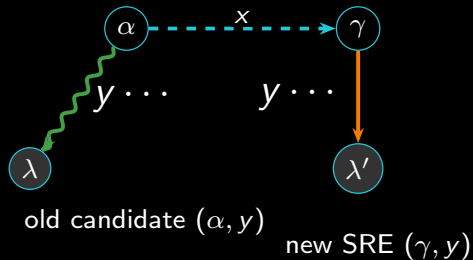
setting: update T to $x \cdot T$

- ▀ λ : newest leaf of old T
 - ▀ α : lowest ancestor of λ with an x -link
 - ▀ Weiner's algorithm finds α and $\gamma = W_x(\alpha)$
 - ▀ inserts a new leaf λ'
- ⇒ Weiner's update already identifies the main nodes needed for SRE maintenance



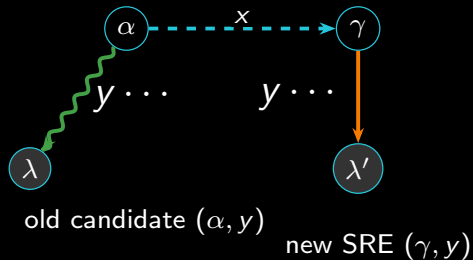
hard-link case

- ▮ $W_x(\alpha) = \gamma$ is an existing node



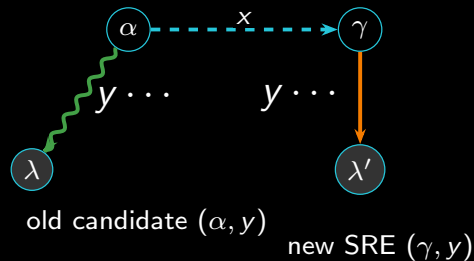
hard-link case

- ▮ $W_x(\alpha) = \gamma$ is an existing node
- ▮ new descending y -edge appears at γ



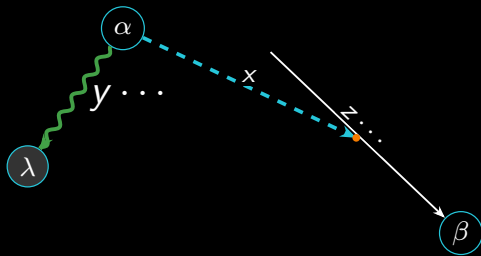
hard-link case

- ▮ $W_x(\alpha) = \gamma$ is an existing node
- ▮ new descending y -edge appears at γ
- ▮ add the new SRE (γ, y)
- ▮ if (α, y) was an SRE: delete it



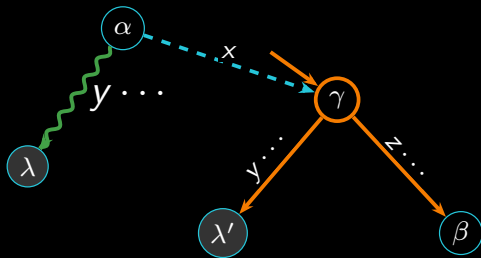
soft-link case

- ▮ $W_x(\alpha)$ points inside an edge



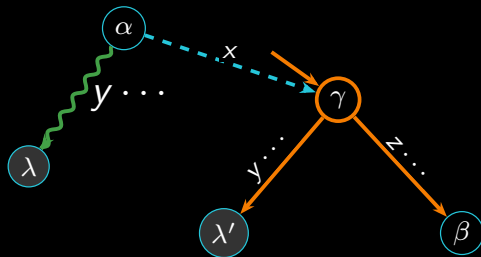
soft-link case

- ▮ $W_x(\alpha)$ points inside an edge
- ▮ Weiner's update splits the edge and creates the new internal node γ



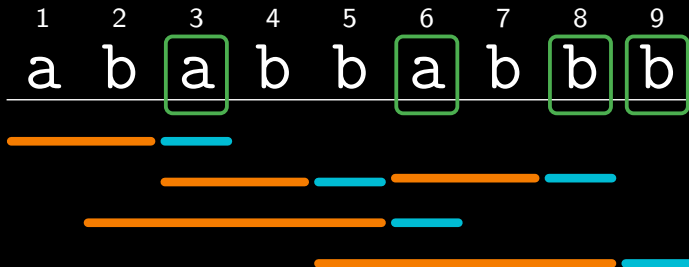
soft-link case

- $W_x(\alpha)$ points inside an edge
- Weiner's update splits the edge and creates the new internal node γ
- The two descending edges of γ create new SREs (γ, y) and (γ, z)
- Delete (α, y) and/or (α, z) if they were SREs



maintaining the rightmost-position SSS

- ▮ each SRE corresponds to a unique suffix-tree edge
- ▮ choose one end position of an occurrence, e.g. the rightmost one
- ▮ when an SRE is inserted or deleted, flip the corresponding bit (positions are stored relative to the right end)
- ▮ unchanged SREs keep their rightmost occurrence when prepending a letter



algorithmic summary

1. run the near-real-time Weiner suffix-tree update
2. receive α, γ, λ' and the relevant first edge letter y (and z in the soft-link case)
3. update only the $O(1)$ SREs that can change
4. update the rightmost-position bit vector together with the SRE set

not shown here, but in the paper:

- ▮ left-to-right online algorithm with the same complexities
- ▮ omitted since it requires more technical details

take-away

Theorem

We can maintain the SRE set and an SSS consisting of rightmost end positions in **worst-case** $O(t_{\text{ST}})$ time per letter, where t_{ST} is the time for a suffix-tree update

- ▮ for polynomial-size integer alphabets, $t_{\text{ST}} = O(\lg^2 \lg n)$
 - ▮ if $\sigma = O(\lg^{1/4} n)$, this becomes $O(\lg \lg n)$
-
- ▮ an SSS is a repetitiveness-based position set, and its minimum size is the number of SREs
 - ▮ right-to-left input matches Weiner's suffix-tree construction
 - ▮ only a constant number of SREs can change in one round
 - ▮ $\Rightarrow$ near-real-time suffix-tree maintenance yields near-real-time SRE/SSS maintenance

thank you for your attention

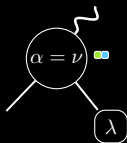
existing literature computing SSSs

work	result
Depuydt+'23	construction time left open
Cenzato+'24	$O(n)$ time
Bonizzoni+'26	$O(n)$ time
Olivares-Navarro'26	$O(n)$ time
Fujimaru+'26	$O(\lg \sigma)$ time amortized per letter
Bonizzoni+'26	$O((\lg n / \lg \lg n)^2)$ worst-case time per letter
this talk	$O(\lg^2 \lg n)$ worst-case time per letter

⇒ first work computing suffixient sets in near-real-time

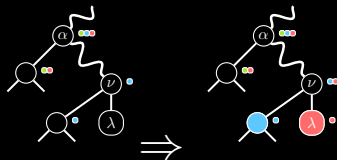
Four update cases in the left-to-right framework

Case 1: $\nu = \alpha$

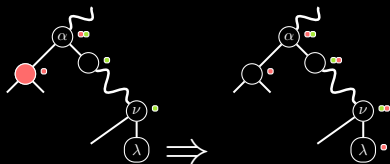


if $\nu = \alpha$, no SRE changes

Case 2: add (ν, x)



Case 3: delete (α, x)



Case 4: delete (μ, y)



Filled nodes indicate SRE witnesses, and colored squares denote the set of letters with defined W-links at that node.